

LAB 03

Digital RTL Design

Sequential Circuit Design

Marwan Abdelfattah

1 ALU

1.1 RTL Design

```
1 module ALU(  
2     input rst, clk,  
3     input [3:0] A, B,  
4     input [1:0] opcode,  
5     output reg [7:0] out  
6 );  
7  
8 reg [3:0] A_reg, B_reg;  
9 reg [1:0] opcode_reg;  
10  
11 always @(posedge clk or posedge rst) begin  
12     if(rst)begin  
13         A_reg <= 0;  
14         B_reg <= 0;  
15         opcode_reg <= 0;  
16     end else begin  
17         A_reg <= A;  
18         B_reg <= B;  
19         opcode_reg <= opcode;  
20     end  
21 end  
22 always @(posedge clk or posedge rst) begin  
23     if(rst)begin  
24         out <= 0;  
25     end else begin  
26         case (opcode)  
27             2'b00: out <= A_reg + B_reg;  
28             2'b01: out <= A_reg * B_reg;  
29             2'b10: out <= A_reg | B_reg;  
30             2'b11: out <= A_reg & B_reg;  
31         endcase  
32     end  
33 end  
34 endmodule
```

1.2 Testbench

```

1 module alu_tb();
2     reg rst, clk;
3     reg [3:0] A, B;
4     reg [1:0] opcode;
5     reg [7:0] out_expected;
6     wire [7:0] out ;
7     ALU DUT (rst,clk,A,B,opcode, out);
8     initial begin
9         clk = 0;
10        forever begin
11            #1 clk = ~clk;
12        end
13    end
14    integer i;
15    initial begin
16        A = 4'd5;
17        B = 4'd3;
18        opcode = 0;
19        out_expected = 0;
20        rst = 1;
21        if(out != 1'b0)begin
22            $display("ERROR - ALU OUTPUT IS INCORRECT");
23            $stop;
24        end
25        @(negedge clk);
26        rst = 0;
27        for(i=0;i<20;i=i+1)begin
28            A = $random();
29            B = $random();
30            opcode = $random();
31            case (opcode)
32                2'b00: out_expected <= A + B;
33                2'b01: out_expected <= A * B;
34                2'b10: out_expected <= A | B;
35                2'b11: out_expected <= A & B;
36            endcase
37            repeat (2) @(negedge clk);
38            if(out != out_expected) begin
39                $display("ERROR - ALU OUTPUT IS INCORRECT");
40                $stop;
41            end
42        end
43        $stop;
44    end
45    initial begin
46        $monitor("Rst=%d, A=%d, B=%d, opcode= %b, out_expected=%d, out=%d",
47            rst, A, B, opcode, out_expected, out);
48    end
49 endmodule

```

1.3 Simulation Waveform

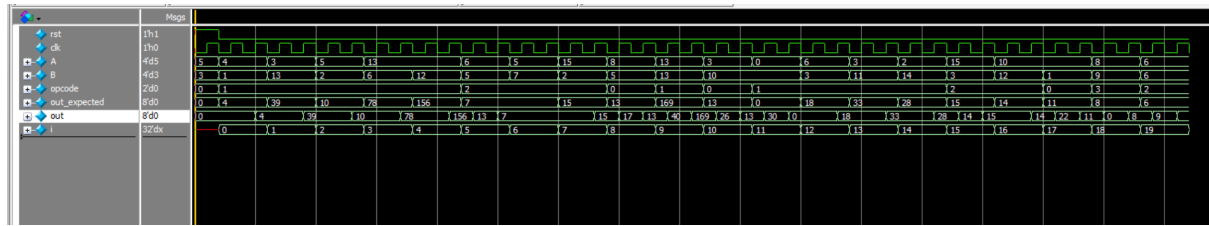


Figure 1: Simulation waveform

```
# Rst=1, A= 5, B= 3, opcode= 00, out_expected= 0, out= 0
# Rst=0, A= 4, B= 1, opcode= 01, out_expected= 4, out= 0
# Rst=0, A= 4, B= 1, opcode= 01, out_expected= 4, out= 4
# Rst=0, A= 3, B=13, opcode= 01, out_expected= 39, out= 4
# Rst=0, A= 3, B=13, opcode= 01, out_expected= 39, out= 39
# Rst=0, A= 5, B= 2, opcode= 01, out_expected= 10, out= 39
# Rst=0, A= 5, B= 2, opcode= 01, out_expected= 10, out= 10
# Rst=0, A=13, B= 6, opcode= 01, out_expected= 78, out= 10
# Rst=0, A=13, B= 6, opcode= 01, out_expected= 78, out= 78
# Rst=0, A=13, B=12, opcode= 01, out_expected=156, out= 78
# Rst=0, A=13, B=12, opcode= 01, out_expected=156, out=156
# Rst=0, A= 6, B= 5, opcode= 10, out_expected= 7, out=156
# Rst=0, A= 6, B= 5, opcode= 10, out_expected= 7, out= 13
# Rst=0, A= 6, B= 5, opcode= 10, out_expected= 7, out= 7
# Rst=0, A= 5, B= 7, opcode= 10, out_expected= 7, out= 7
# Rst=0, A=15, B= 2, opcode= 10, out_expected= 15, out= 7
# Rst=0, A=15, B= 2, opcode= 10, out_expected= 15, out= 15
# Rst=0, A= 8, B= 5, opcode= 00, out_expected= 13, out= 15
# Rst=0, A= 8, B= 5, opcode= 00, out_expected= 13, out= 17
# Rst=0, A= 8, B= 5, opcode= 00, out_expected= 13, out= 13
# Rst=0, A=13, B=13, opcode= 01, out_expected=169, out= 13
# Rst=0, A=13, B=13, opcode= 01, out_expected=169, out= 40
# Rst=0, A=13, B=13, opcode= 01, out_expected=169, out=169
# Rst=0, A= 3, B=10, opcode= 00, out_expected= 13, out=169
# Rst=0, A= 3, B=10, opcode= 00, out_expected= 13, out= 26
# Rst=0, A= 3, B=10, opcode= 00, out_expected= 13, out= 13
# Rst=0, A= 0, B=10, opcode= 01, out_expected= 0, out= 13
# Rst=0, A= 0, B=10, opcode= 01, out_expected= 0, out= 30
# Rst=0, A= 0, B=10, opcode= 01, out_expected= 0, out= 0
# Rst=0, A= 6, B= 3, opcode= 01, out_expected= 18, out= 0
# Rst=0, A= 6, B= 3, opcode= 01, out_expected= 18, out= 18
# Rst=0, A= 3, B=11, opcode= 01, out_expected= 33, out= 18
# Rst=0, A= 3, B=11, opcode= 01, out_expected= 33, out= 33
# Rst=0, A= 2, B=14, opcode= 01, out_expected= 28, out= 33
# Rst=0, A= 2, B=14, opcode= 01, out_expected= 28, out= 28
# Rst=0, A=15, B= 3, opcode= 10, out_expected= 15, out= 28
# Rst=0, A=15, B= 3, opcode= 10, out_expected= 15, out= 14
# Rst=0, A=15, B= 3, opcode= 10, out_expected= 15, out= 15
# Rst=0, A=10, B=12, opcode= 10, out_expected= 14, out= 15
# Rst=0, A=10, B=12, opcode= 10, out_expected= 14, out= 14
# Rst=0, A=10, B= 1, opcode= 00, out_expected= 11, out= 14
# Rst=0, A=10, B= 1, opcode= 00, out_expected= 11, out= 22
# Rst=0, A=10, B= 1, opcode= 00, out_expected= 11, out= 11
# Rst=0, A= 8, B= 9, opcode= 11, out_expected= 8, out= 11
# Rst=0, A= 8, B= 9, opcode= 11, out_expected= 8, out= 0
# Rst=0, A= 8, B= 9, opcode= 11, out_expected= 8, out= 8
```

Figure 2: Transcript

2 Up-Down Counter

```
1 module udcounter #(
2     parameter width = 8
3 ) (
4     input clk, rst,
5     input [width-1 : 0] load,
6     input load_en,
7     input up_ndown,
8     output reg [width-1:0] cnt
9 );
10
11 always @(posedge clk) begin
12     if(!rst)begin
13         cnt <= 0;
14     end else if (load_en) begin
15         cnt <= load;
16     end else if (up_ndown)begin
17         cnt <= cnt + 1;
18     end else begin
19         cnt <= cnt - 1;
20     end
21 end
22 endmodule
```

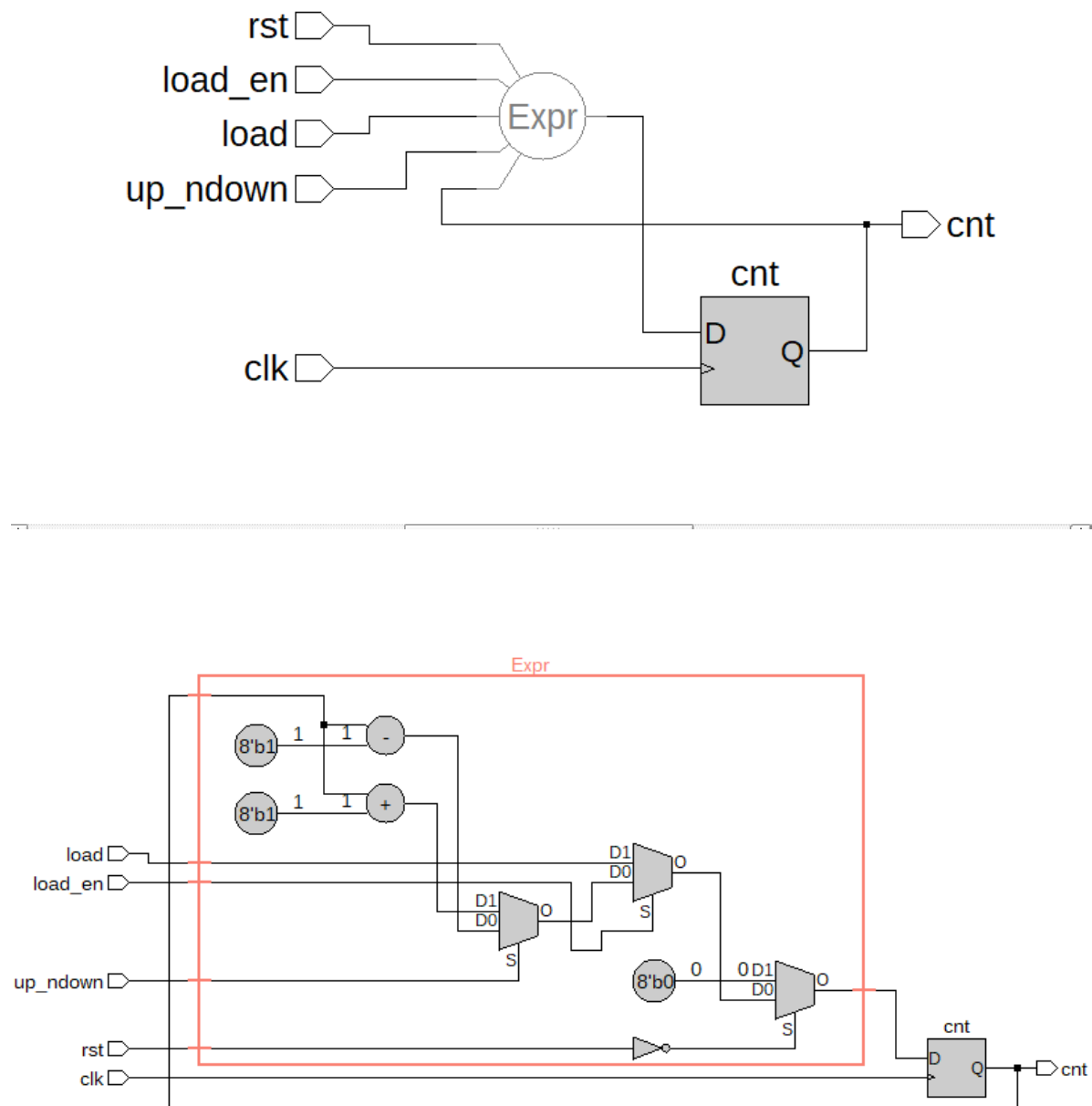


Figure 3: Questa Lint Schematic Diagram

3 8-bit Shift-Left/Shift-Right Register

```
1 module shift_left_right(  
2     input clk, rst, SI, LEFT_RIGHT,  
3     output reg [7:0] P0  
4 );  
5  
6 always @(posedge clk or posedge rst) begin  
7     if(rst) begin  
8         P0 <= 0;  
9     end else if (LEFT_RIGHT) begin  
10        P0 <= {SI,P0[7:1]};  
11    end else begin  
12        P0 <= {P0[6:0] , SI};  
13    end  
14 end  
15 endmodule
```

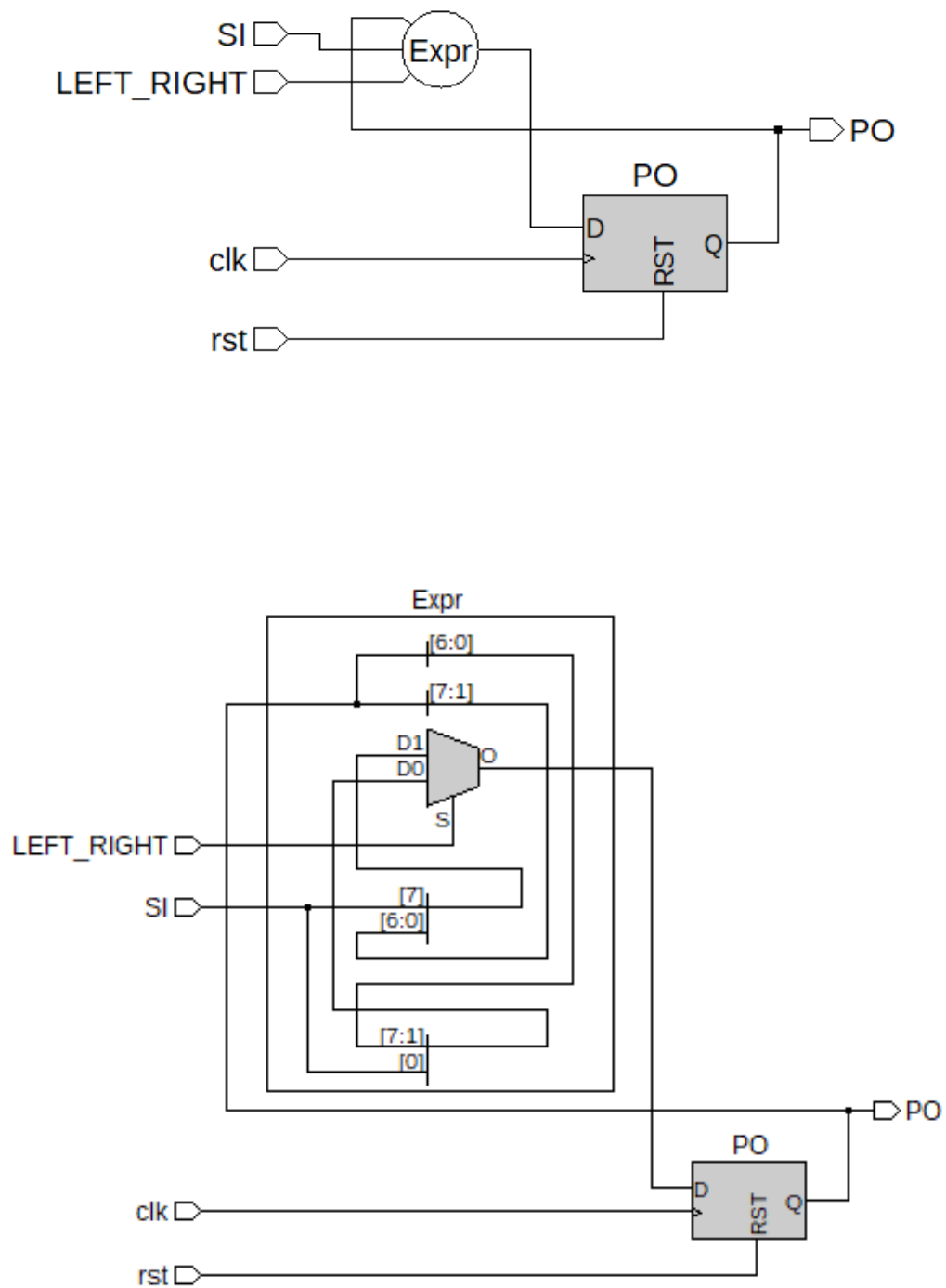


Figure 4: Questa Lint Schematic Diagram